Real-Time Data & Subscription — Full Notes

Django + Django REST Framework + Channels + Redis + Celery

Level: Beginner to Advanced

Language: Bangla + English

Project: Real-Time Notification & Order Tracking System

Chapter 1: Introduction to Real-Time Systems

1.1 What Is Real-Time Data?

Real-time data হলো এমন data যা তৈরি বা update হওয়ার পর খুব অল্প সময়ের মধ্যে ব্যবহারকারীর কাছে পৌঁছে যায়।

উদাহরণ:

- Chat messages
- Live notifications
- Order status updates
- Stock-price updates
- Live dashboards
- Online user status
- Delivery tracking
- Multiplayer game events

ধরা যাক, একটি e-commerce website-এ user order করল। Order status `processing` থেকে `shipped` হলো। User-কে page refresh করতে না দিয়ে server সরাসরি update পাঠাতে পারে।

1.2 What Is a Subscription?

Subscription হলো কোনো নির্দিষ্ট event, channel, topic বা resource-এর update পাওয়ার জন্য client-এর subscribe করা।

উদাহরণ:

```
User A subscribes to personal notifications.
User B subscribes to chat room 10.
Admin subscribes to live order updates.
Client subscribes to stock symbol ABC.
```

কোনো event publish হলে সংশ্লিষ্ট subscriber-রা সেটি পেতে পারে।

1.3 Real-Time vs Traditional HTTP

Traditional HTTP

```
Browser
   |
   | HTTP Request
   v
Django Server
   |
   | HTTP Response
   v
Browser
```

সাধারণ HTTP request-response flow-তে server response দেওয়ার পরে একই request-এর মাধ্যমে পরবর্তী update পাঠাতে পারে না।

WebSocket

```
Browser <================> Django ASGI Server
             WebSocket
```

WebSocket connection খোলা থাকলে client ও server উভয়েই message পাঠাতে পারে।

1.4 Polling vs Long Polling vs WebSocket vs SSE

Technology	Communication	Best use case
Polling	Client বারবার request করে	Simple status checks
Long polling	Server response প্রস্তুত না হওয়া পর্যন্ত request অপেক্ষা করে	Legacy real-time updates
WebSocket	Two-way, persistent	Chat, live interaction

	connection	
SSE	Server থেকে client-এ one-way stream	Notifications, live feeds

Recommendation: Chat এবং bidirectional interaction-এর জন্য WebSocket ভালো starting point। শুধু server থেকে browser-এ event পাঠাতে হলে Server-Sent Events (SSE)-ও বিবেচনা করা যায়।

Chapter 2: Understand the Technology Stack

2.1 Django

Django application-এর প্রধান দায়িত্ব:

- HTTP APIs
- Authentication
- Authorization
- Database operations
- Business logic
- Data validation

2.2 Django REST Framework (DRF)

DRF দিয়ে REST API তৈরি করা যায়।

উদাহরণ:

```
POST /api/orders/
GET  /api/orders/101/
GET  /api/notifications/
```

DRF নিজে WebSocket server নয়।

2.3 Django Channels

Django Channels Django-কে ASGI-এর মাধ্যমে WebSocket connection এবং asynchronous event handling সমর্থন করতে সাহায্য করে।

এর প্রধান components:

- Consumer

- Routing
- Channel layer
- Groups
- Authentication middleware

2.4 Redis

Redis একটি in-memory data store, যা বিভিন্ন কাজে ব্যবহার করা যায়:

- Channels channel layer
- Celery broker
- Celery result backend
- Cache
- Pub/Sub messaging

একটি Redis server একাধিক কাজে ব্যবহার করা সম্ভব। তবে production-এ memory, failure isolation, persistence এবং access control বিবেচনা করে পৃথক instance ব্যবহার করা যেতে পারে।

2.5 Celery

Celery background task execute করে।

উদাহরণ:

- Email পাঠানো
- Report তৈরি করা
- Order process করা
- External API call
- Bulk data processing

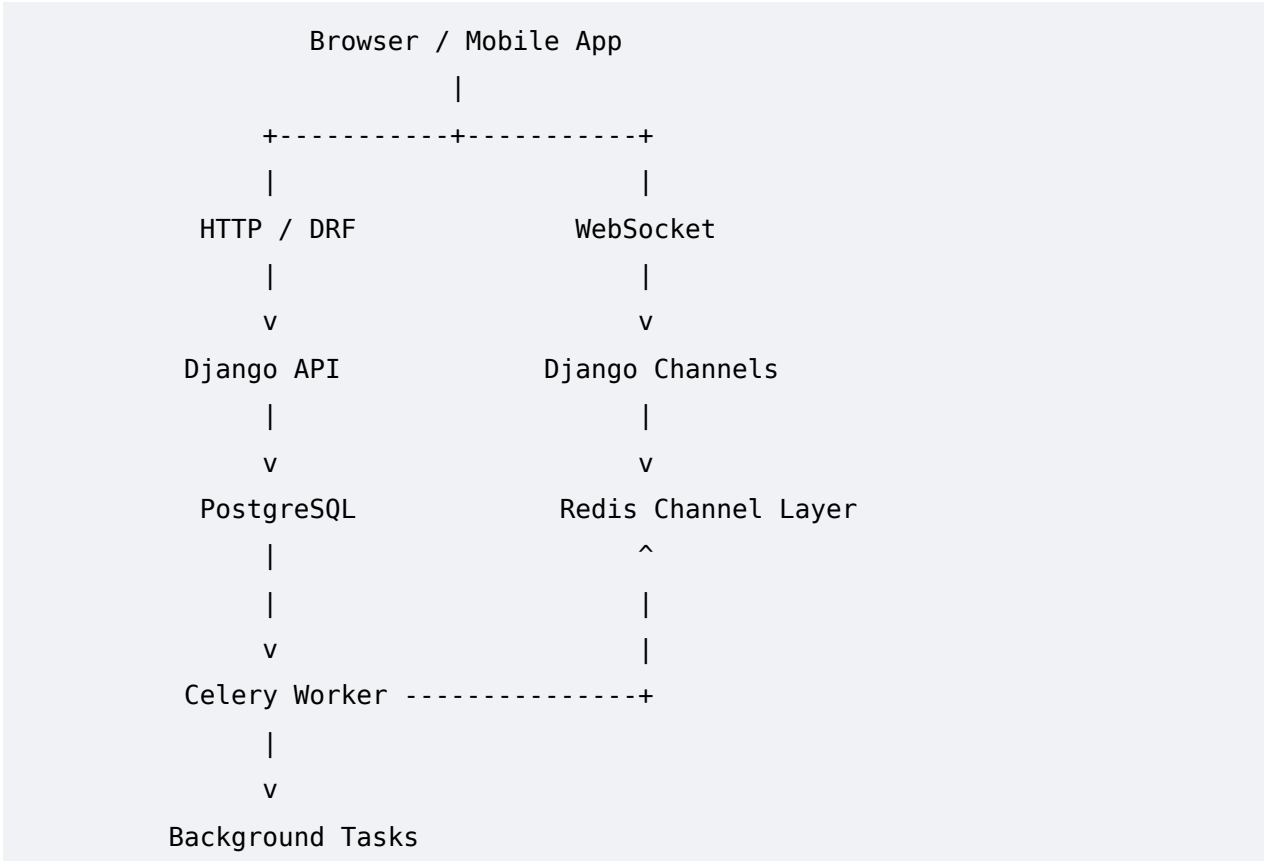
Celery নিজে browser-এর সঙ্গে WebSocket connection তৈরি করে না।

2.6 PostgreSQL

PostgreSQL-এ persistent business data রাখা হয়:

- Users
- Orders
- Messages
- Notifications
- Payments
- Subscription records

2.7 Complete Architecture



Architecture-এর মূল ধারণা:

- HTTP/DRF: data তৈরি ও retrieve করে।
- PostgreSQL: business data স্থায়ীভাবে রাখে।
- Celery: background processing করে।
- Channels: connected clients-এ event পাঠায়।
- Redis channel layer: application instance-গুলোর মধ্যে event route করতে সাহায্য করে।

Chapter 3: WebSocket Fundamentals

3.1 What Is a WebSocket?

WebSocket হলো একটি communication protocol, যা client ও server-এর মধ্যে persistent, two-way connection তৈরি করে।

WebSocket URL:

```
ws://localhost:8000/ws/notifications/
```

Secure WebSocket:

```
wss://example.com/ws/notifications/
```

Production HTTPS website-এর ক্ষেত্রে সাধারণত `wss://` ব্যবহার করা উচিত।

3.2 WebSocket Lifecycle

```
CONNECTING
    |
    v
OPEN
    |
    | Send / Receive messages
    |
    v
CLOSING
    |
    v
CLOSED
```

Connection fail হতে পারে, server restart হতে পারে, অথবা network disconnect হতে পারে। তাই client-side reconnect logic প্রয়োজন হতে পারে।

3.3 Basic JavaScript Example

```
const socket = new WebSocket(
    "ws://localhost:8000/ws/notifications/"
);

socket.onopen = () => {
    console.log("Connected");
};

socket.onmessage = (event) => {
    console.log("Received:", event.data);
};

socket.onclose = () => {
    console.log("Disconnected");
};

socket.onerror = (error) => {
    console.error("WebSocket error:", error);
};
```

Message পাঠাতে:

```
socket.send(
    JSON.stringify({
        type: "ping"
    })
);
```

Server message receive করার জন্য তৈরি না থাকলে এই message process হবে না। Protocol client ও server—দুই পাশেই define করতে হয়।

Chapter 4: Install and Configure Django Channels

4.1 Install Packages

```
pip install django djangorestframework channels channels-redis celery redis
```

একটি নতুন project তৈরি করতে:

```
django-admin startproject config .
python manage.py startapp notifications
```

Project structure:

```
project/
├── manage.py
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── celery.py
└── notifications/
    ├── __init__.py
    ├── models.py
    ├── views.py
    ├── consumers.py
    ├── routing.py
    ├── tasks.py
    └── realtime.py
```

4.2 Start Redis

Docker ব্যবহার করলে:

```
docker run -d \
  --name redis \
  -p 127.0.0.1:6379:6379 \
  redis:7
```

PowerShell-এ একই command এক লাইনে:

```
docker run -d --name redis -p 127.0.0.1:6379:6379 redis:7
```

Connection test:

```
docker exec -it redis redis-cli ping
```

Expected output:

```
PONG
```

এই setup development-এর জন্য। Production-এ Redis authentication, private networking, monitoring এবং persistence requirements configure করতে হবে।

4.3 Update `settings.py`

```
INSTALLED_APPS = [
    # Default Django apps...
    "channels",
    "rest_framework",
    "notifications",
]

ASGI_APPLICATION = "config.asgi.application"

CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [
                "redis://127.0.0.1:6379/2"
            ],
        },
    },
}
```

`channels_redis` production ব্যবহারের জন্য বহুল ব্যবহৃত channel layer backend।

Chapter 5: ASGI Configuration

5.1 What Is ASGI?

ASGI-এর পূর্ণরূপ Asynchronous Server Gateway Interface।

WSGI মূলত synchronous HTTP application serving-এর জন্য পরিচিত। ASGI HTTP-এর পাশাপাশি WebSocket-এর মতো asynchronous protocol handling-ও সমর্থন করে।

5.2 Configure `config/asgi.py`

```
import os

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings",
)

from django.core.asgi import get_asgi_application

django_asgi_app = get_asgi_application()

from channels.auth import AuthMiddlewareStack
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.security.websocket import (
    AllowedHostsOriginValidator,
)

from notifications.routing import websocket_urlpatterns

application = ProtocolTypeRouter({
    "http": django_asgi_app,

    "websocket": AllowedHostsOriginValidator(
        AuthMiddlewareStack(
            URLRouter(websocket_urlpatterns)
        )
    ),
})
```

এই configuration-এ:

- `ProtocolTypeRouter` HTTP ও WebSocket আলাদা করে।
- `AuthMiddlewareStack` Django session authentication সরবরাহ করে।
- `URLRouter` WebSocket URL-এর সঙ্গে consumer map করে।

- `AllowedHostsOriginValidator` WebSocket origin validation-এ সাহায্য করে।

`ALLOWED_HOSTS` সঠিকভাবে configure করতে হবে। Cookie-based authentication ব্যবহার করলে origin validation, HTTPS এবং cookie security গুরুত্বপূর্ণ।

Chapter 6: Create a WebSocket Consumer

6.1 What Is a Consumer?

Consumer হলো Django Channels-এর component, যা connection, message এবং disconnect event handle করে।

6.2 Create `notifications/consumers.py`

```
from channels.generic.websocket import AsyncJsonWebsocketConsumer


class NotificationConsumer(AsyncJsonWebsocketConsumer):

    async def connect(self):
        await self.accept()

        await self.send_json({
            "type": "connection",
            "message": "Connected successfully",
        })

    async def disconnect(self, close_code):
        pass

    async def receive_json(self, content, **kwargs):
        if content.get("type") == "ping":
            await self.send_json({
                "type": "pong",
            })
```

এটি একটি basic demonstration consumer। এখানে যে কেউ connection করতে পারে, তাই private notification system-এ এই version ব্যবহার করা উচিত নয়।

6.3 Consumer Methods

Method	Purpose

`connect()`	Connection শুরু হলে
`disconnect()`	Connection বন্ধ হলে
`receive()`	Text বা binary message গ্রহণ
`receive_json()`	JSON message গ্রহণ
`send()`	Text message পাঠানো
`send_json()`	JSON message পাঠানো
`group_add()`	Group-এ channel যুক্ত করা
`group_discard()`	Group থেকে channel সরানো

Chapter 7: WebSocket Routing

Create `notifications/routing.py`:

```
from django.urls import path
```

```
from .consumers import NotificationConsumer

websocket_urlpatterns = [
    path(
        "ws/notifications/",
        NotificationConsumer.as_asgi(),
    ),
]
```

WebSocket routing এবং Django HTTP URL routing আলাদা।

HTTP URL:

```
# config/urls.py

from django.contrib import admin
from django.urls import path

urlpatterns = [
    path("admin/", admin.site.urls),
]
```

WebSocket URL `config/asgi.py` -তে `URLRouter` -এর মাধ্যমে configure করা হয়েছে।

Chapter 8: Real-Time Subscription with Groups

8.1 What Is a Group?

Channels group হলো একাধিক connected channel-এর logical collection। Group-এ event পাঠালে সেই group-এর connected channels event পেতে পারে।

উদাহরণ:

```
user_10
chat_25
order_101
admin_dashboard
```

Group নিজে persistent database subscription record নয়।

8.2 Private Notification Consumer

Create `notifications/consumers.py` :

```
from channels.generic.websocket import AsyncJsonWebsocketConsumer


```

```
class NotificationConsumer(AsyncJsonWebsocketConsumer):

    async def connect(self):
        user = self.scope["user"]

        if not user.is_authenticated:
            await self.close()
            return

        self.group_name = f"user_{user.pk}"

        await self.channel_layer.group_add(
            self.group_name,
            self.channel_name,
        )

        await self.accept()

    async def disconnect(self, close_code):
        if hasattr(self, "group_name"):
            await self.channel_layer.group_discard(
                self.group_name,
                self.channel_name,
            )

    async def notification_message(self, event):
        await self.send_json({
            "type": "notification",
            "message": event["message"],
        })
```

এখানে:

- Authenticated user ছাড়া connection গ্রহণ করা হচ্ছে না।
- User-এর ID দিয়ে private group নির্ধারণ করা হচ্ছে।
- Disconnect হলে group membership সরানো হচ্ছে।
- `notification_message()` incoming channel-layer event-কে WebSocket JSON message-এ রূপান্তর করে।

`AuthMiddlewareStack` ব্যবহার করলে `self.scope["user"]` সাধারণত session authentication থেকে পাওয়া যায়।

8.3 Important Security Rule

Client থেকে এই ধরনের message গ্রহণ করে সরাসরি private group join করাবে না:

```
{
  "user_id": 99
}
```

এতে user অন্য কারও notification subscribe করার চেষ্টা করতে পারে।

Private group-এর পরিচয় authenticated user এবং server-side authorization থেকে নির্ধারণ করতে হবে।

Chapter 9: Publish Real-Time Events

9.1 Create `notifications/realtime.py`

```
from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer


def send_user_notification(user_id, message):
    channel_layer = get_channel_layer()

    async_to_sync(channel_layer.group_send)(
        f"user_{user_id}",
        {
            "type": "notification.message",
            "message": message,
        },
    )
```

9.2 Call the Function

```
send_user_notification(
    user_id=10,
    message="Your order has been shipped.",
)
```

যদি user-এর WebSocket connection active থাকে, event তার connection-এ পাঠানো হতে পারে।

Important: Group-এ কেউ connected না থাকলেও application event publish করতে পারে; Channels group কোনো durable notification inbox নয়।

9.3 Event Structure

```
{
    "type": "notification.message",
```

```
    "message": "Your order has been shipped.",
}
```

Event-এর `type` consumer method-এ route হয়:

```
async def notification_message(self, event):
    await self.send_json({
        "message": event["message"],
    })
```

Chapter 10: Frontend Subscription

10.1 Connect to WebSocket

```
const socket = new WebSocket(
    "ws://localhost:8000/ws/notifications/"
);

socket.onopen = () => {
    console.log("Subscribed to notifications");
};

socket.onmessage = (event) => {
    const data = JSON.parse(event.data);

    if (data.type === "notification") {
        console.log(data.message);
    }
};

socket.onclose = () => {
    console.log("Connection closed");
};
```

এখানে authenticated browser session cookie server-এ পাঠাতে হবে। Cross-origin setup হলে credentials ও allowed origins ঠিকভাবে configure করতে হবে।

10.2 Reconnect Strategy

Production frontend-এ connection drop হলে retry করা যেতে পারে।

```
function connectWebSocket() {
    const socket = new WebSocket(
        "wss://example.com/ws/notifications/"
```

```
    );

    socket.onopen = () => {
        console.log("Connected");
    };

    socket.onmessage = (event) => {
        console.log(JSON.parse(event.data));
    };

    socket.onclose = () => {
        setTimeout(connectWebSocket, 3000);
    };

    socket.onerror = () => {
        socket.close();
    };
}

connectWebSocket();
```

এটি একটি basic example। Production-এ exponential backoff, maximum retry, visibility handling এবং duplicate event handling বিবেচনা করা উচিত।

Chapter 11: Real-Time Notification Model

শুধু WebSocket message পাঠালেই notification history তৈরি হয় না। স্থায়ী notification history-এর জন্য database model ব্যবহার করো।

11.1 `notifications/models.py`

```
from django.conf import settings
from django.db import models


class Notification(models.Model):

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="notifications",
    )
```

```
    title = models.CharField(max_length=200)
    message = models.TextField()

    is_read = models.BooleanField(default=False)

    created_at = models.DateTimeField(
        auto_now_add=True
    )

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return self.title
```

11.2 Run Migrations

```
python manage.py makemigrations
python manage.py migrate
```

11.3 Create Notification and Publish Event

```
from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer
from django.db import transaction

from .models import Notification


def create_notification(user, title, message):
    with transaction.atomic():
        notification = Notification.objects.create(
            user=user,
            title=title,
            message=message,
        )

        transaction.on_commit(
            lambda: publish_notification(notification)
        )

    return notification
```

```
def publish_notification(notification):
    channel_layer = get_channel_layer()

    async_to_sync(channel_layer.group_send)(
        f"user_{notification.user_id}",
        {
            "type": "notification.message",
            "notification_id": notification.pk,
            "title": notification.title,
            "message": notification.message,
        },
    )
```

Consumer-কে update করো:

```
async def notification_message(self, event):
    await self.send_json({
        "type": "notification",
        "id": event["notification_id"],
        "title": event["title"],
        "message": event["message"],
    })
```

এখানে database record আগে commit হয়, তারপর event publish করার চেষ্টা হয়।

তবে database commit এবং Redis publish আলাদা operation। Commit-এর পরে Redis publish ব্যর্থ হলে notification record থাকবে, কিন্তু live event miss হতে পারে। নির্ভরযোগ্য event delivery-এর জন্য transactional outbox pattern বিবেচনা করো।

Chapter 12: Django REST Framework Notification APIs

12.1 Serializer

Create `notifications/serializers.py`:

```
from rest_framework import serializers

from .models import Notification


class NotificationSerializer(serializers.ModelSerializer):

    class Meta:
        model = Notification
```

```
        fields = [
            "id",
            "title",
            "message",
            "is_read",
            "created_at",
        ]
        read_only_fields = [
            "id",
            "title",
            "message",
            "created_at",
        ]
```

12.2 List User Notifications

Create `notifications/views.py`:

```
from rest_framework import generics, permissions

from .models import Notification
from .serializers import NotificationSerializer


class NotificationListView(generics.ListAPIView):

    serializer_class = NotificationSerializer
    permission_classes = [
        permissions.IsAuthenticated
    ]

    def get_queryset(self):
        return Notification.objects.filter(
            user=self.request.user
        )
```

12.3 URL Configuration

Create `notifications/urls.py`:

```
from django.urls import path

from .views import NotificationListView

urlpatterns = [
```

```
    path(
        "",
        NotificationListView.as_view(),
        name="notification-list",
    ),
]
```

Update `config/urls.py`:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path(
        "api/notifications/",
        include("notifications.urls"),
    ),
]
```

Authenticated user নিজের notification list দেখতে পারবে:

```
GET /api/notifications/
```

Production-এ pagination এবং প্রয়োজন অনুযায়ী read/unread filtering যোগ করো।

Chapter 13: Celery + Real-Time Notifications

Celery background processing করে, আর Channels event publish করে connected clients-এ update পাঠায়।

13.1 Configure `config/celery.py`

```
import os

from celery import Celery

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings",
)

app = Celery("config")

app.config_from_object(
```

```
    "django.conf:settings",
    namespace="CELERY",
)

app.autodiscover_tasks()
```

Update `config/__init__.py`:

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

13.2 Configure `settings.py`

```
CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"

CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/1"

CELERY_ACCEPT_CONTENT = ["json"]

CELERY_TASK_SERIALIZER = "json"

CELERY_RESULT_SERIALIZER = "json"

CELERY_TIMEZONE = "Asia/Dhaka"

CELERY_ENABLE_UTC = True
```

Production-এ Redis credentials environment variable বা secret manager থেকে নিতে হবে।

13.3 Create a Task

Create `notifications/tasks.py`:

```
from celery import shared_task

from .models import Notification
from .realtime import send_user_notification


@shared_task(
    autoretry_for=(ConnectionError, TimeoutError),
    retry_backoff=True,
    max_retries=3,
)
def process_order_notification(
```

```
    user_id,
    order_id,
):
    message = (
        f"Your order {order_id} has been processed."
    )

    notification = Notification.objects.create(
        user_id=user_id,
        title="Order Update",
        message=message,
    )

    send_user_notification(
        user_id=user_id,
        message=message,
    )

    return {
        "notification_id": notification.pk,
        "status": "completed",
    }
```

এটি একটি teaching example। এখানে task retry হলে duplicate notification তৈরি হতে পারে। Production-এ unique event ID, idempotency checks এবং appropriate transaction handling যোগ করতে হবে।

13.4 Run the Worker

```
celery -A config worker --loglevel=INFO
```

13.5 Trigger the Task

```
process_order_notification.delay(
    user_id=10,
    order_id=101,
)
```

Flow:

```
Django API
    |
    v
Celery Broker (Redis)
```

```
    |
    v
Celery Worker
    |
    v
Save Notification
    |
    v
Channels Group Send
    |
    v
Redis Channel Layer
    |
    v
Connected Browser
```

Chapter 14: Real-Time Chat System

Chat system-এর জন্য subscription সাধারণত room-based হয়।

```
chat_1
chat_2
chat_3
```

14.1 Chat Model

```
from django.conf import settings
from django.db import models


class ChatRoom(models.Model):
    name = models.CharField(max_length=100)


class Message(models.Model):
    room = models.ForeignKey(
        ChatRoom,
        on_delete=models.CASCADE,
        related_name="messages",
    )

    sender = models.ForeignKey(
        settings.AUTH_USER_MODEL,
```

```
        on_delete=models.CASCADE,
    )

    content = models.TextField()

    created_at = models.DateTimeField(
        auto_now_add=True
    )

    class Meta:
        ordering = ["created_at"]
```

14.2 Room Consumer Design

একটি chat consumer-এ:

1. Authenticated user যাচাই করতে হবে।
2. Room-এর membership যাচাই করতে হবে।
3. User-কে room group-এ যোগ করতে হবে।
4. Incoming message validate করতে হবে।
5. Database-এ message save করতে হবে।
6. Room group-এ event publish করতে হবে।
7. Disconnect হলে group membership সরাতে হবে।

Pseudo-flow:

```
Client connects to room 25
          |
          v
Authenticate user
          |
          v
Check room membership
          |
          v
Join chat_25
          |
          v
Receive message
          |
          v
Validate and save
          |
```

```
          v
Broadcast to chat_25
```

Security: Room ID জানলেই যেন যে কেউ private room-এ ঢুকতে না পারে। প্রতিটি connection ও subscription-এ membership যাচাই করো।

Chapter 15: Subscription Types

15.1 Personal Subscription

```
group_name = f"user_{user.pk}"
```

Use cases:

- Personal notifications
- Account alerts
- Order updates

15.2 Room Subscription

```
group_name = f"chat_{room_id}"
```

Use cases:

- Group chat
- Team collaboration
- Live discussion

Room membership server-side যাচাই করতে হবে।

15.3 Resource Subscription

```
group_name = f"order_{order_id}"
```

Use cases:

- Order tracking
- Report status
- Job progress

15.4 Dashboard Subscription

```
group_name = "admin_dashboard"
```

এ ধরনের group-এ join করার আগে admin permission যাচাই করতে হবে।

15.5 Important Group Rules

- Group names সীমিত length-এর মধ্যে রাখো।
- User input সরাসরি group name হিসেবে ব্যবহার করবে না।
- Sensitive event public group-এ পাঠাবে না।
- Group membership-কে permanent database subscription মনে করবে না।
- Offline delivery প্রয়োজন হলে database-এ event history রাখো।

Chapter 16: Redis Pub/Sub vs Channels vs Redis Streams

Feature	Redis Pub/Sub	Channels Groups	Redis Streams
Main purpose	Publish/subscribe messaging	Application-level event routing	Persistent event stream
Connected client broadcast	Application code প্রয়োজন	Supported through channel layer	Application code প্রয়োজন
Offline replay	No	Not a durable history system	Supported through retained entries
Consumer groups	No	Not the same model	Supported
Best use	Lightweight messaging	WebSocket event delivery	Durable event processing patterns

Redis Pub/Sub message disconnected subscriber-এর জন্য রেখে দেয় না। Redis Streams event history এবং consumer-group-based processing-এর জন্য বেশি উপযোগী হতে পারে।
Channels group হলো Django Channels-এর abstraction; এটি Redis Pub/Sub-এর সঙ্গে পুরোপুরি একই feature set নয়।

Chapter 17: Reliability and Event Delivery

17.1 At-Most-Once vs At-Least-Once

Real-time system-এ event delivery নিয়ে সচেতন হতে হবে।

- **At-most-once:** event হারাতে পারে, কিন্তু duplicate delivery এড়ানোর চেষ্টা করে।
- **At-least-once:** retry বা redelivery-এর কারণে duplicate event আসতে পারে।
- **Exactly-once:** পুরো distributed system-এ নিখুঁত exactly-once effect নিশ্চিত করা কঠিন।

Celery task এবং WebSocket event-কে একবারই execute বা deliver হবে—এমন নিশ্চয়তা ধরে নেওয়া উচিত নয়।

17.2 Event IDs

```
{
  "event_id": "evt_123",
  "type": "order.updated",
  "order_id": 101,
  "status": "shipped"
}
```

Client `event_id` ব্যবহার করে duplicate event detect করতে পারে। Business-level idempotency server-এও implement করা উচিত।

17.3 Sequence Numbers and Timestamps

```
{
  "event_id": "evt_124",
  "sequence": 12,
  "type": "order.updated",
  "status": "delivered",
  "timestamp": "2026-10-22T12:00:00Z"
}
```

Sequence number দিয়ে event ordering বা missing updates শনাক্ত করা সহজ হতে পারে। Timestamp একা সবসময় total ordering guarantee করে না।

17.4 Reconnect and Resynchronization

WebSocket disconnect হলে client কিছু event miss করতে পারে।

Recommended flow:

1. Client WebSocket connect করে।
2. Server live event পাঠায়।
3. Connection drop হয়।
4. Client reconnect করে।
5. Client HTTP API থেকে latest state বা missed event history retrieve করে।
6. Client আবার live updates গ্রহণ করে।

গুরুত্বপূর্ণ business event-এর ক্ষেত্রে শুধু WebSocket-এর ওপর নির্ভর করবে না।

17.5 Transactional Outbox Pattern

যখন database update এবং event publishing দুটোই নির্ভরযোগ্যভাবে প্রয়োজন হয়:

1. একই database transaction-এ business record এবং outbox event save করো।
2. Background worker pending outbox records retrieve করবে।
3. Event publish করবে।
4. সফল publication track করবে।
5. Retry এবং duplicate handling implement করবে।

এতে database commit এবং broker publish-এর মধ্যকার failure window মোকাবিলা করা সহজ হয়। তবুও idempotency ও operational monitoring প্রয়োজন।

Chapter 18: Performance and Scaling

18.1 Common Bottlenecks

- Too many open connections
- Large event payloads
- Slow database queries
- Excessive group broadcasts
- High Redis memory usage
- Celery queue backlog
- Too many simultaneous reconnects

18.2 Optimization

- Event payload ছোট রাখো।
- বড় data পাঠানোর পরিবর্তে resource ID পাঠিয়ে client-কে API থেকে details retrieve করতে দাও।
- Database indexes ও query performance ঠিক রাখো।

- প্রয়োজন অনুযায়ী pagination ব্যবহার করো।
- Celery queue workload অনুযায়ী আলাদা করো।
- Redis latency, memory ও connection count monitor করো।
- ASGI workers এবং connection capacity load test করে tune করো।

18.3 Horizontal Scaling

একাধিক Django ASGI instance থাকলেও shared channel layer-এর মাধ্যমে event route করা যায়।

```
                Load Balancer
                     |
           +---------+----------+
           |                    |
           v                    v
      ASGI App 1            ASGI App 2
           |                    |
           +---------+----------+
                     |
                     v
                Redis Layer
                     ^
                     |
                Celery Worker
```

Production deployment-এ WebSocket proxy support, idle timeout, graceful shutdown, Redis availability এবং worker capacity নিশ্চিত করতে হবে।

Chapter 19: Security Checklist

- Authentication ছাড়া private WebSocket access বন্ধ রাখো।
- প্রতিটি room/resource subscription-এ authorization যাচাই করো।
- WebSocket origin validation configure করো।
- Production-এ `wss://` ব্যবহার করো।
- Incoming payload validate করো।
- Message size সীমিত রাখো।
- Rate limiting এবং abuse protection বিবেচনা করো।
- Sensitive data group broadcast-এ পাঠিও না।
- Redis public internet-এ expose করবে না।
- Broker credentials secure রাখো।
- Client-কে user ID দিয়ে অন্য user-এর private group select করতে দেবে না।

- Disconnect, token expiration এবং permission revocation-এর behavior design করো।

Chapter 20: Testing

20.1 Test WebSocket Connection

Django Channels-এর `WebsocketCommunicator` ব্যবহার করে asynchronous consumer test করা যায়।

```
from channels.testing import WebsocketCommunicator
from django.test import SimpleTestCase

from config.asgi import application


class WebSocketTests(SimpleTestCase):

    async def test_connection(self):
        communicator = WebsocketCommunicator(
            application,
            "/ws/notifications/",
        )

        connected, _ = await communicator.connect()

        self.assertTrue(connected)

        await communicator.disconnect()
```

এটি শুধু connection behavior test করার উদাহরণ। Private notification consumer-এ authentication দরকার হলে authenticated scope বা test client setup করতে হবে।

20.2 Test Cases

- Authenticated user connect করতে পারে।
- Anonymous user private connection পায় না।
- User নিজের notification পায়।
- অন্য user-এর private event পায় না।
- Invalid payload reject হয়।
- Disconnect হলে group membership পরিষ্কার হয়।
- Duplicate task business data duplicate করে না।
- Redis unavailable হলে error handling ঠিক থাকে।

- Reconnect-এর পরে client latest state retrieve করতে পারে।

Chapter 21: Deployment Checklist

Development

- Redis চালু
- Django development server
- Channels configuration
- Celery worker
- Browser WebSocket client

Production

- PostgreSQL database
- Redis channel layer
- Secure Celery broker configuration
- ASGI server
- Reverse proxy ও WebSocket upgrade support
- HTTPS/WSS
- Worker supervision ও graceful shutdown
- Logs, metrics এবং alerting
- Redis memory monitoring
- Connection limits
- Authentication ও authorization
- Retry, idempotency ও durable business state

Celery Beat ব্যবহার করলে periodic task scheduler-এর জন্য উপযুক্ত deployment এবং duplicate scheduler protection-ও প্রয়োজন।

Chapter 22: Troubleshooting

WebSocket connects, but no messages arrive

Check:

- Event-এর group name সঠিক কি না।
- Consumer event type-এর সঙ্গে handler method মেলে কি না।
- Redis channel layer reachable কি না।
- Worker event publish করছে কি না।

- User সঠিক group-এ যুক্ত হয়েছে কি না।

Anonymous users can connect

Check:

- `AuthMiddlewareStack` configured কি না।
- Browser session cookie পাঠাচ্ছে কি না।
- Consumer authentication check করছে কি না।
- Token authentication হলে middleware token যাচাই করছে কি না।

Works locally but not in production

Check:

- Reverse proxy WebSocket upgrade support
- HTTPS এবং `wss://`
- Redis hostname ও network access
- Allowed hosts ও origin validation
- ASGI deployment
- Worker ও application logs
- Load balancer idle timeout

Notifications are missing after disconnect

এটি WebSocket-এর স্বাভাবিক সীমাবদ্ধতা হতে পারে। Database-backed notification history এবং reconnect-এর পরে synchronization implement করো।

Duplicate notifications appear

Check:

- Celery retry
- Duplicate task execution
- Multiple scheduler instances
- Duplicate client event processing
- Event idempotency

Chapter 23: Interview Questions

Q1. What is real-time data?

এমন data যা update হওয়ার পর খুব অল্প সময়ের মধ্যে client-এর কাছে পৌঁছে যায়।

Q2. What is a subscription?

কোনো channel, room, topic বা resource-এর update পাওয়ার জন্য client-এর আগ্রহ নিবন্ধন বা subscribe করা।

Q3. What is WebSocket?

একটি protocol যা client ও server-এর মধ্যে persistent, bidirectional connection দেয়।

Q4. What is Django Channels?

Django application-এ WebSocket ও asynchronous protocol handling-এর জন্য ব্যবহৃত framework।

Q5. What is ASGI?

Python application ও asynchronous server-এর মধ্যে interface; WebSocket-এর মতো protocol handling সমর্থন করে।

Q6. What is Redis channel layer?

Channels application-এর বিভিন্ন consumer বা process-এর মধ্যে event communicate করার backend।

Q7. What is the difference between Celery and Channels?

Celery background task execute করে। Channels WebSocket connection ও event routing পরিচালনা করে।

Q8. Does WebSocket store message history?

না। WebSocket transport connection; history-এর জন্য database বা durable event storage প্রয়োজন।

Q9. Can Redis Pub/Sub guarantee offline delivery?

না। Disconnected subscriber message miss করতে পারে।

Q10. Why use `transaction.on_commit()`?

Database transaction commit হওয়ার পরে event publish করার চেষ্টা করতে, যাতে worker অসম্পূর্ণ database state না দেখে।

Q11. What is a Channels group?

একাধিক connected channel-এর logical collection, যেখানে event broadcast করা যায়।

Q12. Why is authorization important in subscriptions?

একজন user যেন অন্য user-এর private messages, orders বা notifications access করতে না পারে।

Q13. How do you handle reconnects?

Retry strategy, authentication refresh এবং reconnect-এর পরে API বা event history থেকে state synchronization দিয়ে।

Q14. How do you scale WebSocket applications?

Shared channel layer, multiple ASGI instances, load balancer, resource limits এবং monitoring ব্যবহার করে।

Q15. When should Redis Streams be considered?

যখন retained events, replay বা consumer-group-based processing প্রয়োজন হয়।

Chapter 24: Practice Projects

Beginner

1. WebSocket connection demo
2. Live clock dashboard
3. Real-time notification popup
4. Online/offline status

Intermediate

1. Django notification API
2. Group chat
3. Celery background email + live status
4. Live order tracking
5. Admin dashboard updates

Advanced

1. Multi-room chat with membership authorization
2. Reliable notification system with database history
3. Celery + Channels + transactional outbox
4. Live job-progress dashboard
5. Reconnect and event replay mechanism
6. Multi-instance WebSocket deployment
7. Real-time event monitoring system

Chapter 25: Complete Learning Roadmap

1. HTTP request-response
2. Django views ও DRF APIs
3. Redis fundamentals
4. JavaScript WebSocket API
5. ASGI

6. Django Channels consumers
7. WebSocket routing
8. Channel layers ও groups
9. Authentication ও authorization
10. Database-backed notifications
11. Celery integration
12. Event retry ও idempotency
13. Reconnection ও synchronization
14. Redis Pub/Sub বনাম Streams
15. Monitoring ও testing
16. Docker এবং production deployment

Final Summary

Component	Responsibility
Django	Business logic ও web application
DRF	REST APIs
PostgreSQL	Persistent business data
WebSocket	Live bidirectional communication

Django Channels	WebSocket connection ও event routing
Redis channel layer	Application instances-এর মধ্যে event delivery
Celery	Background task execution
Celery Beat	Scheduled task publishing
Redis Streams	Retained event stream ও consumer-group processing

Final project goal: Django REST Framework দিয়ে order create করো, PostgreSQL-এ order status save করো, Celery দিয়ে background processing করো, Django Channels দিয়ে subscribed client-কে live status পাঠাও, এবং disconnect হলে client-কে API থেকে latest status পুনরায় synchronize করতে দাও।

এই architecture real-time chat, notifications, order tracking, live dashboards এবং asynchronous backend workflows শেখার জন্য শক্ত ভিত্তি তৈরি করবে।